

Modelagem Relacional de Dados

Parte 2 — Modelagem Lógica (do ER às tabelas)

Unidade 4 · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Nesta aula

- Do **modelo conceitual (ER)** ao **modelo lógico (tabelas)**
- **Regras de tradução:** entidades, atributos, relacionamentos
- **Atributos multivalorados/compostos e entidades fracas**
- Relacionamentos **1:1, 1:N, M:N**
- **Ferramentas: Mermaid** (padrão, diagrama como código), também draw.io e brModelo — e SQL Power Architect na VM

Do conceitual ao lógico

O que traduz em quê

Conceitual (ER)	Lógico (relacional)
Entidade	Tabela
Atributo	Coluna
Atributo determinante	Chave Primária (PK)
Relacionamento	Chave Estrangeira (FK) ou tabela associativa

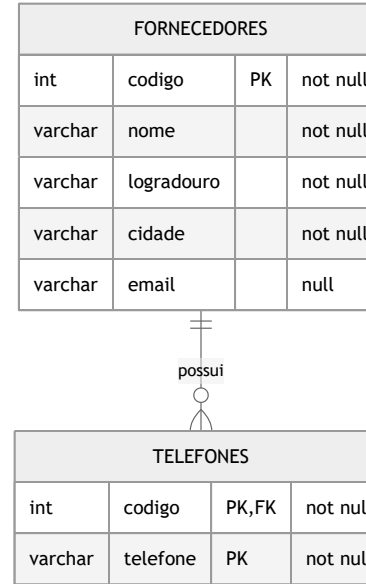
💡 O modelo lógico ainda é independente do fabricante — só na modelagem **física** escolhemos tipos e recursos de um SGBD.

Tradução E-R → Tabelas — Entidade e multivalorado

Diagrama E-R

FORNECEDORES

Tabelas (lógico)

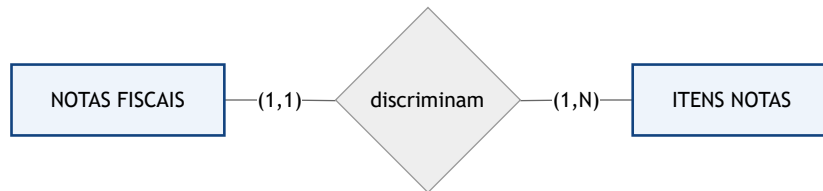


A entidade vira **tabela**; o **composto** ENDERECO é decomposto em colunas; o **multivalorado** {telefones} vira a tabela TELEFONES (com FK).

 **SQL Power Architect:** defina tipos e constraints na aba *Columns* (PK, NOT NULL); use *Add Relationship* para ligar TELEFONES.

Tradução E-R → Tabelas — Relacionamento 1:N

Diagrama E-R



Tabelas (lógico)

NOTAS_FISCAIS			
int	numero	PK	not null
date	data		not null
numeric	valor		not null



ITENS_NF			
int	numero	PK,FK	not null
int	item	PK	not null
int	quantidade		not null
numeric	valor		not null

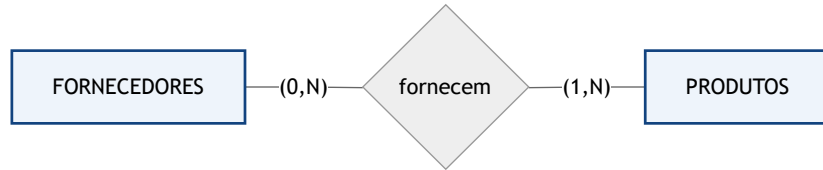
A PK do lado "1" (numero) vira **FK** em ITENS_NF ; a PK de ITENS é **composta** (numero + item).



SQL Power Architect: arraste de uma tabela para a outra — o sistema sugere a cardinalidade automaticamente.

Tradução E-R → Tabelas — Relacionamento N:M

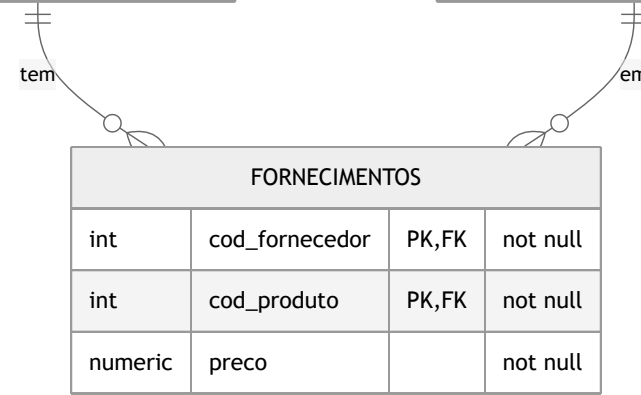
Diagrama E-R



Tabelas (lógico)

FORNECEDORES			
int	codigo	PK	not null
varchar	nome		not null

PRODUTOS			
int	codigo	PK	not null
varchar	nome		not null



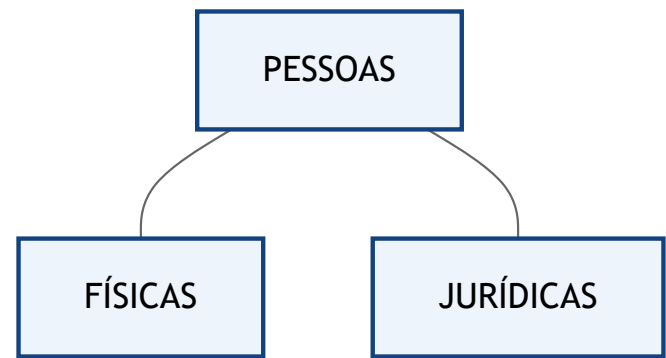
O M:N vira a **tabela associativa** FORNECIMENTOS : as duas FKs formam a **PK composta**, mais o atributo preco .



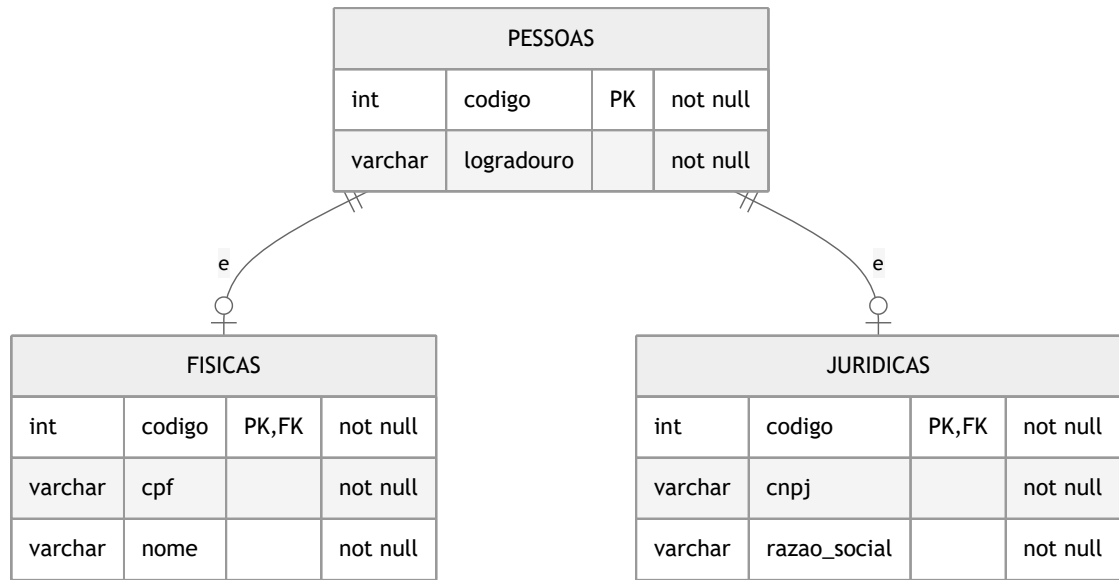
SQL Power Architect: para N:M, crie a tabela intermediária *manualmente* e ligue com dois relacionamentos 1:N.

Tradução E-R → Tabelas — Generalização

Diagrama E-R



Tabelas (lógico)

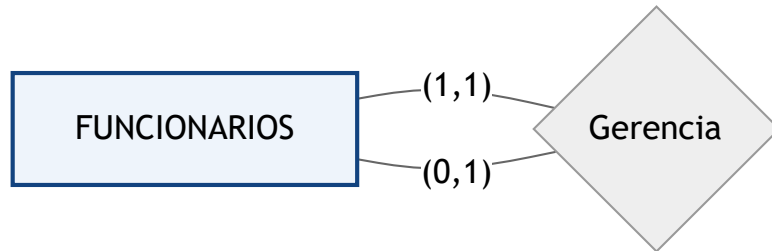


Uma **tabela por tipo**: FISICAS e JURIDICAS usam a mesma PK de PESSOAS (que também é **FK**) — herdam a identidade.

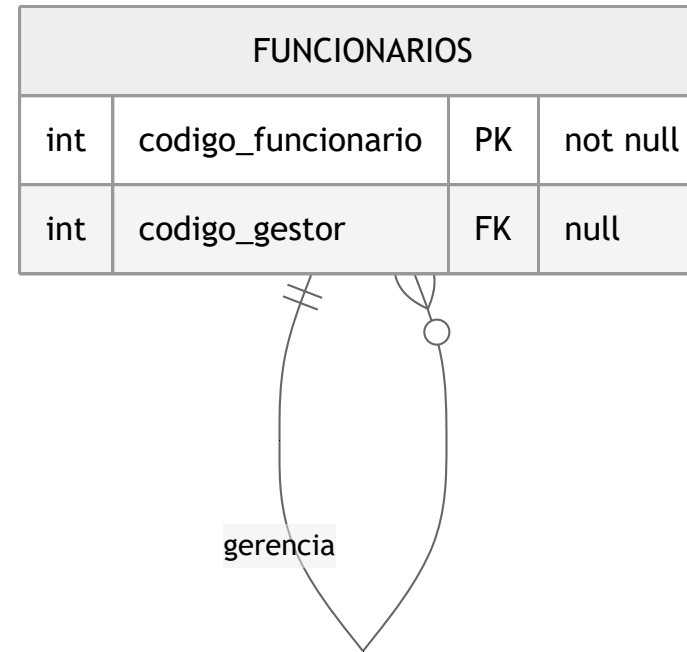
 **SQL Power Architect**: crie uma tabela por subtipo e marque a PK do supertipo como *PK* e *FK* na subtabela.

Tradução E-R → Tabelas — Auto-relacionamento

Diagrama E-R



Tabelas (lógico)



`codigo_gestor` é uma **FK** que aponta para a própria tabela **FUNCIONARIOS** (o gestor também é funcionário).

 **SQL Power Architect:** adicione a FK `codigo_gestor` referenciando a própria **FUNCIONARIOS** (auto-relacionamento).

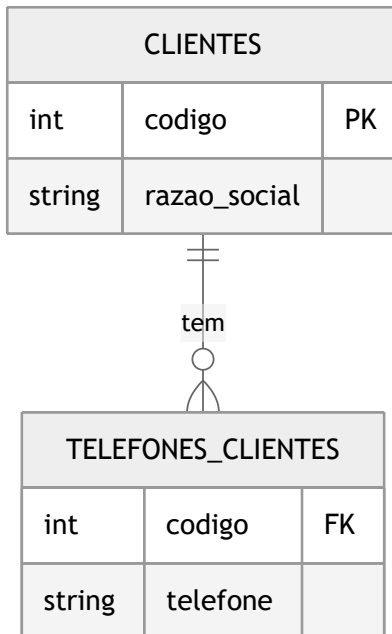
Entidades regulares

- Cada entidade vira uma **tabela**.
- O **atributo determinante** vira a **PK**.
- Atributos **simples e monovalorados** viram **colunas**.

```
CREATE TABLE clientes (  
    codigo          INT          PRIMARY KEY,  
    razao_social    VARCHAR(100) NOT NULL,  
    logradouro      VARCHAR(100) NOT NULL  
);
```

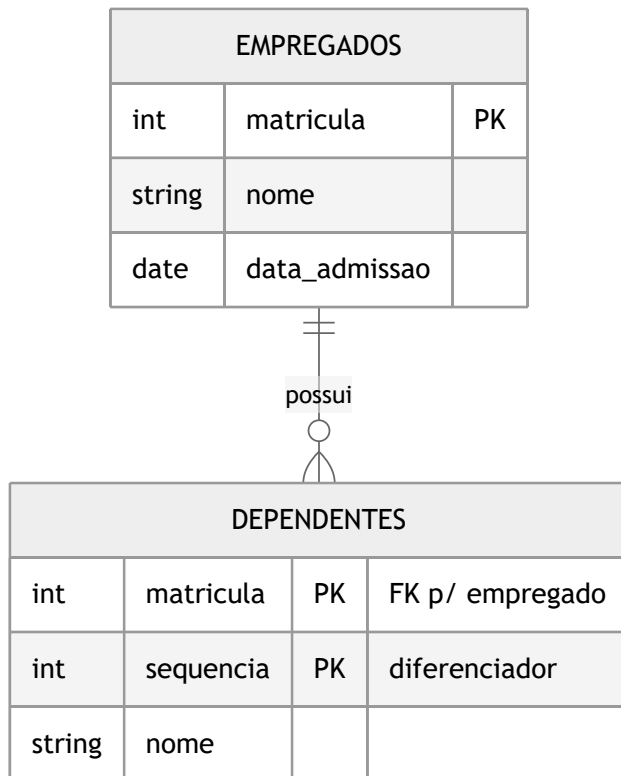
Atributos multivalorados e compostos

- **Multivalorado** (ex.: vários telefones) → **nova tabela** ligada por FK.
- **Composto** (ex.: endereço = logradouro + número + cidade) → **decompor** em colunas simples.



Entidades fracas

- Não têm identidade própria — dependem de uma entidade forte.
- Viram tabela com **PK composta**: a chave da entidade forte + um atributo diferenciador.



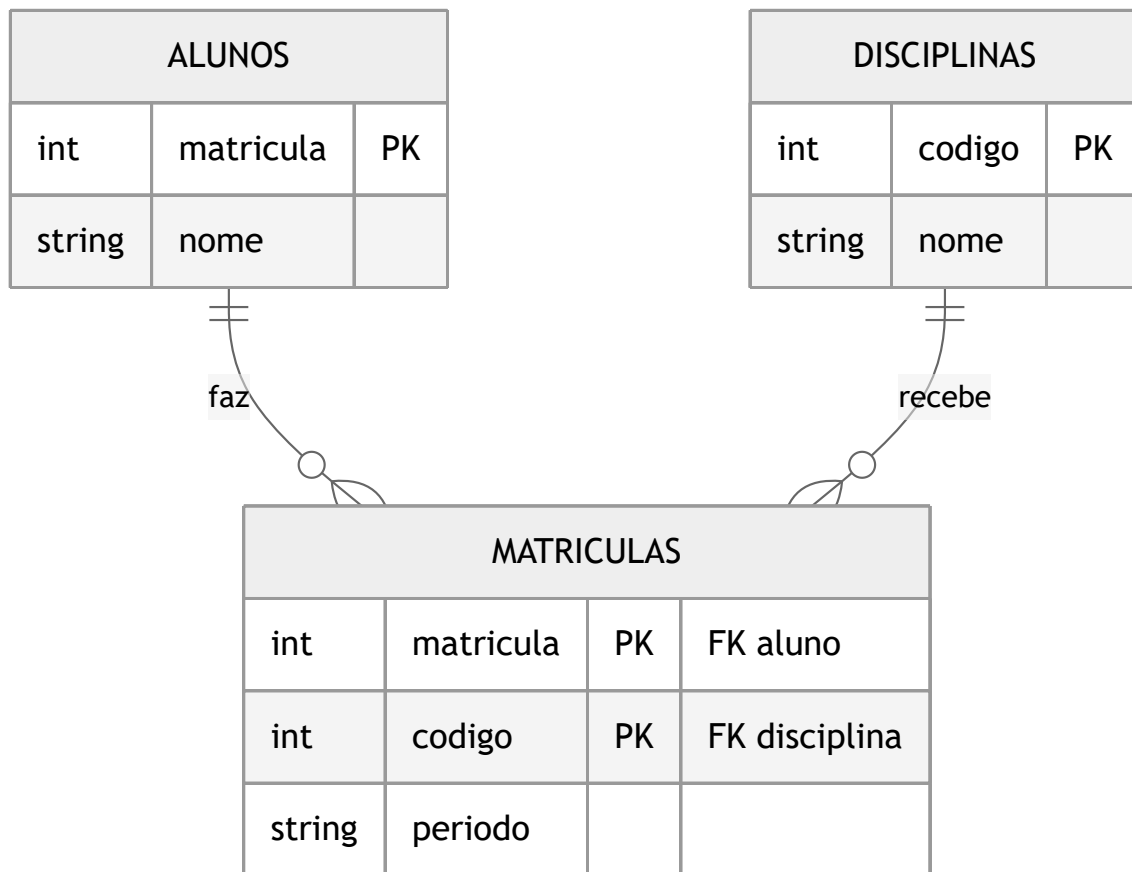
Relacionamentos 1:1 e 1:N

- **1:1** — a PK de um lado vira **FK** no outro (onde fizer mais sentido).
- **1:N** — a PK do lado "1" vira **FK** na tabela do lado "N".

```
-- 1:N - cada oferta pertence a uma disciplina
ALTER TABLE ofertas
  ADD CONSTRAINT fk_oferta_disc
  FOREIGN KEY (disciplina) REFERENCES disciplinas(codigo);
```

Relacionamento M:N → tabela associativa

Todo **muitos-para-muitos** vira uma **tabela associativa** com as duas FKs (que juntas formam a PK):



Notação: Mermaid (diagrama como código)

O próprio diagrama vira **texto versionável** — os deste material são feitos assim:

```
erDiagram
    ALUNOS ||--o{ MATRICULAS : faz
    DISCIPLINAS ||--o{ MATRICULAS : recebe
    ALUNOS { int matricula PK }
    MATRICULAS { int matricula PK int codigo PK }
```

Ponta (Mermaid)	Significado
--o	zero ou um
--	exatamente um
--o{	zero ou muitos
-- {	um ou muitos

Mermaid × SQL Power Architect

Mermaid — quando...

- projeto **versionado** no Git
- documentação integrada (Markdown)
- equipe colaborativa
- diagramas simples a médios

SQL Power Architect — quando...

- modelos **grandes/complexos**
- **engenharia reversa** de um banco
- geração de DDL visual
- disponível na **VM LabDatabase**



Ambas as ferramentas estão na VM; use a que melhor se encaixa ao tamanho do projeto.



Padrão da disciplina: Mermaid. Também aceitos: **draw.io** e **brModelo** (mesma notação crow's foot)
— entregue a imagem/PDF e, no Mermaid, também o código.

Exercício

Pegue os ERs que você modelou na Parte 1 (**Controle de Pedidos** e **Jogos/RPG**) e:

1. Traduza cada entidade em **tabela** (defina PK).
2. Resolva os relacionamentos com **FK** ou **tabela associativa**.
3. Trate **multivalorados/compostos** e **entidades fracas**.
4. Escreva o **DDL** (`CREATE TABLE ...`) — continua na **Unidade 5**.

Bibliografia

- COUGO, P. **Modelagem Conceitual e Projeto de Bancos de Dados**. Campus, 1997.
- HEUSER, C. A. **Projeto de Banco de Dados**. 6ª ed. Bookman, 2009.
- ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7ª ed. Pearson, 2019.

Dúvidas?

howard.cruz@faesa.br

Próximo →

Modelagem Lógica — Aprofundando